K1.pdf

Sanjay Gupta

January 2025

1 Group

Sanjay Gupta — s347gupt — 20885923
Yibo Ma — y439ma — 20883952

- Please note that we are a group from K2 onwards.

2 Gitlab

Please locate the repository at: `https://git.uwaterloo.ca/s347gupt/CS452/-/tree/A1?ref_type=heads`

The submission hash to be used: b4ec7e8e0657911df349646a3c54b077cf150419

3 Kernel Structure

3.1 Data Structures

3.1.1 Ring Buffer

The code utilizes ring buffers to function as queues (FIFO). Ring buffer is defined using templates (in A0, they were used to pass commands to the Marklin hardware). For the purposes of K1, the ring buffer is used for storing ints (mapping between indices and array of task descriptors). In the future, this mapping can be eliminated by updating the ringbuffer class/template to account for pointers to task descriptors.

3.1.2 Priority Queues

Basic implementation (KISS) for min-heap priority queue, used exclusively by the kernel scheduler for determining which task to run. The class creates a static array (of size HEAP_SIZE), (since no heap), and updates a size field on push/pop operations. Similar to the Ring Buffer class, it is a template but implemented with ints (mapping).

3.2 Classes

The code is organized into classes to enforce encapsulation and reduce coupling between separate processes.

3.2.1 Task Descriptor

Task descriptors isolate logic associated with typical functions needed by any running task; able to do the following:

- Create new tasks (setting themselves as the parent)
- Setting/Returning values associated with tasks (ex: returning TID)
- Each task descriptor contains a context structure: each is 64 bit int: SP, SPSR, ELR, and registers X0-X30. In order to save kernel state on context switches, the address of the context structure used by the kernel is passed into X0 and the task context structure goes to X1.

NOTE: Tasks are only issued to the kernel if memory is available for the task

3.2.2 Memory

Memory for tasks is allocated after the image, at 0x90000. Each stack is allocated 4 MB, and there can be a maximum of 64 tasks. The rationale behind this was that we have approximately 1 GB (minus 76 MB for videocore) after the end of image to work with, and having a task stack of size 256 MB is well within this region.

3.2.3 Kernel

- The kernel has several methods (they should be made private in a future revision), such as Create, MyTid, MyParentTid, Yield, Exit. These functions should only be called by the kernel's handler function (aside from boostraping FirstUserTask).
- The Kernel uses the memory class for allocating stacks to new tasks (called from create); a ring buffer for popping (create) and pushing (exit) available task ids; a pointer to the active task descriptor (last running task before entering kernel mode); an array of task descriptors (of size MAX_TASKS= 64), and a priority queue for O(1) retrieval of the task of highest priority.
- Scheduler: Pops the id of the highest priorty task from the ready queue, and returns a pointer to the corresponding entry in the task decriptor table.
- DispatchTask: Takes as input a task and sets it's state to active; calls kernel_to_task_asm assembly function to perform a context switch. See more details below.

- Handler: Takes as input the masked output of the ESR register (paramater 'N' passed in with svc call). Based on N, execute priveledged kernel operations (retrieving task information), creating new tasks, returning resources, etc.
- SetRetval: Save the system call return value into the task context X0 register (context.x[0]).

3.2.4 UserTask

We define a usertask class to represent the abstraction of software running in the user level (above the kernel) in EL0. This simply defines two tasks, Task1 (FUT), and Task2. Task1 is referenced in main.cpp as the bootstrapped first task.

3.3 Assembly

3.3.1 kernel_to_task_asm

This is called by the DispatchTask function in order to perform a context switch.

- X0 contains address of kernel_context
- X1 contains address of task_context
- First we store x0 and x1 on the stack, note that sp points to the task_context
- Store all kernel registers, then store kernel SPSR, SP, ELR.
- Load pointer to task_context from stack
- load the registers and processor state
- eret to execute task in EL0

3.3.2 Vector Table

Upon issuing SVC calls, the architecture routes the exception to a vector table, defined in boot.S (boot.S stores the vector table in VBAR_EL1 system register). Each bank (4 exception handlers, 4 banks) routes to dummy routines refined in dummy.h. The one we are interested in is synchronous exceptions from lower EL, AArch64. This is defined in the table to branch to task_to_kernel_asm.

3.3.3 task_to_kernel_asm

- we load the task context pointer from the stack (taking care not to overwrite X0 and X1 (used in CREATE system call). We essentially perform the opposite of kernel_to_task_asm, saving the entire task state, popping the kernel context into X0 and then finally restoring the kernel context.

4 Output

The code prints:
Created: ⟨1⟩
Created: ⟨2⟩
TID: ⟨3⟩, PARENT-TID: ⟨0⟩
TID: ⟨3⟩, PARENT-TID: ⟨0⟩
Created: ⟨3⟩
TID: ⟨4⟩, PARENT-TID: ⟨0⟩
TID: ⟨4⟩, PARENT-TID: ⟨0⟩
Created: ⟨4⟩
FirstUserTask: exiting
TID: ⟨1⟩, PARENT-TID: ⟨0⟩
TID: ⟨2⟩, PARENT-TID: ⟨0⟩
TID: ⟨1⟩, PARENT-TID: ⟨0⟩
TID: ⟨2⟩, PARENT-TID: ⟨0⟩
NO MORE TASKS

FirstUserTask (FUT) is assigned TID 0, so when the bootstrapped task is given control from Dispatch, it calls the CREATE function which executes the svc and passes the code for SVC_CREATE into the syndrome register (save user state, restore kernel context). After the kernel creates the task, it sets the return value for the system call in the task descriptors context. Since this task had lower priority than FUT, the scheduler picks FUT (dispatch → save kernel state, restore user task state) and resumes execution, receiving the tid 1 – since ring buffer for TID's is created by pushing ints from [0-MAX_TASKS), and prints "Created 1". Then FUT issues another create syscall for a low priority task. This results in the same sequence of steps, but a tid of 2 is returned.

At this point, we have two LOW and one MEDIUM priority tasks on the ready queue. When the FUT makes the next create system call, it now passes a HIGH priority flag. When the kernel creates and adds this new task to the ready queue, the scheduler immediately chooses it (over the other three tasks of lower priority). This high priority task is then dispatched. First, it makes a system call to fetch its task id (3), and is immediately reselected by the scheduler since it is the only HIGH priority task. Then it makes another system call to fetch it's parents tid (0). It prints these values, "TID (3), Parent(0)", calls YIELD (switch from task to kernel, add current task (3) back to ready queue), and it is again rescheduled and dispatched immediately. It then prints the line again, and calls the Exit system call, which the kernel uses to reclaim resources (tid, memory block) – note that these are not zeroed out.

Once again, we have two LOW and one MEDIUM priority tasks on the ready queue. The scheduler chooses the FUT. Upon restoring the threads state, the return value from creating the high priority task is returned in X0 (3) and prints "Created 3". This process is repeated for TID: 4, and eventually returns to the FUT where it prints "Created 4". Since there are no more tasks to create, FUT prints "FirstUserTask: exiting" before calling EXIT.

Now we only have two low priority tasks in the ready queue (the scheduler can choose either). When these are invoked, they will have to make two system calls each: one to request its TID and another for its parent TID. The printed lines are the result of returning from the second of each of these calls for the parent id. After printing the line, the task calls YEILD, which places it at the end of the ready queue. This makes the other LOW task selected next. Each of these is chosen to run once more, printing the same line before calling EXIT. Once the last LOW priority task calls EXIT, the ready queue is empty and we stop checking for tasks. (end of task loop in main prints NO MORE TASKS!)